MPTA-Repair: Simultaneous Multi-Property Repair for Networks of Timed Automata via Iterative Solving

Guangtong Zhou, Run Chang, Ji Wu*
Beihang University, Beijing, China
Email: zhougt@buaa.edu.cn, handsomeruncr@gmail.com, wuji@buaa.edu.cn

***Abstract*—Networks of timed automata (NTA) are widely used to model and verify real-time industrial systems, which are typically required to satisfy multiple predefined properties simultaneously. When verification fails, repairing an NTA is challenging because a valid repair must modify erroneous clock constraints, satisfy all properties, and preserve the original functional behavior. We present MPTA-Repair, an automated repair framework for multi-property NTA. It uses an iterative, counterexample-guided MaxSMT approach to generate minimally modified candidates, exploiting relations among properties, counterexamples, and repair variables to optimize search. Each candidate is validated by full-property regression verification and an admissibility check based on functional equivalence. Compared to the TARTAR baseline, MPTA-Repair achieves higher effectiveness on faulty models generated via our proposed mutation strategy. Specifically, it improves the success rate from 72% to 93% on benchmarks derived from UPPAAL and the XtaBenchmarkSuite, and from 20% to 76% on an industrial dataset constructed in this work.**

***Index Terms*—networks of timed automata, automated model repair, MaxSMT, multi-property verification**

I. INTRODUCTION

Real-time systems combine discrete control logic with quantitative timing requirements. In railway control, communication protocols, and aerospace scheduling [1]–[3], safety depends on actions satisfying permitted time bounds; an alarm, message, pulse, actuation, or resource release may be unsafe if it occurs too early, too late, or for an invalid duration.

Timed automata (TAs) model such behavior with locations, clocks, guards, resets, and invariants [4], [5]; practical systems are often modeled as networks of timed automata (NTA). Safety requirements are encoded as verifiable properties, such as bounded response, deadline satisfaction, alarm duration, and deadlock freedom [6]. For an NTA, violating such a property indicates a safety defect whose discovery during implementation or testing can be costly. Clock-bound errors in guards and invariants are a common source of timed-safety violations, since incorrect deadlines, timeouts, or durations can make otherwise valid discrete behavior occur outside its permitted time window [2], [7], [8]. We therefore focus on clock-bound repair, which modifies numerical bounds in guards and invariants. Model checkers such as UPPAAL, Kronos, or opaal can return timed diagnostic traces (TDTs), i.e., timed counterexamples reaching violating states [9]–[11]; these traces expose failures but not which bound is wrong or how to change it.

Repair becomes more challenging in the setting of satisfying multiple properties simultaneously. A bound change that blocks one TDT may leave another violation feasible, violate a previously satisfied property, or compromise the original functional behavior. In this paper, functional behavior refers to the untimed discrete behavior characterized by functional equivalence. The running example in Section II illustrates this effect: repairing an early gear-engagement violation can make an acknowledgement deadline fail. A practical repair method must therefore generate candidate repairs from local counterexamples, but accept them only after full-property regression verification and a TARTAR-style admissibility check based on untimed functional equivalence [7].

Existing repair techniques provide an important foundation. The TARTAR framework [7] establishes a baseline for clock-bound repair by encoding a TDT into linear real arithmetic formulas that capture both path feasibility and the violated property condition. By introducing variation variables for numerical clock bounds, it uses MaxSMT solving to compute small changes to guards and invariants while verifying admissibility check based on functional equivalence. Related work also studies parametric timed automata, clock-guard repair, robustness analysis, and SMT-based repair [12]–[15]. However, these workflows are still commonly driven by one violated property at a time [7]. This single-property style is limiting for industrial systems that must satisfy several properties simultaneously: a local edit that removes one violation can expose another. Because repairs are generated property by property, the coupling among properties and repair variables is not used to guide candidate generation and is exposed only during validation, reducing solver guidance and making globally feasible repairs harder to find.

This paper presents MPTA-Repair, an automated framework for simultaneous multi-property clock-bound repair of NTA via iterative solving. Given an NTA and a property set, MPTA-Repair first performs verification. If at least one property is violated, it searches for changes to numerical bounds in transition guards and location invariants to satisfy all properties and preserve the original functional behavior. The framework maintains a pool of property-trace constraints, uses relations among properties, counterexamples, and repairable bounds to guide MaxSMT-based candidate generation, and validates each candidate by full-property regression verification and an admissibility check based on functional equivalence.

We evaluate MPTA-Repair on faulty models generated by clock-bound mutation. The evaluation includes benchmarks derived from UPPAAL [16] and the XtaBenchmarkSuite [17], and an industrial dataset constructed in this work. Compared to the TARTAR baseline, MPTA-Repair achieves higher effectiveness on faulty models generated via our proposed mutation strategy. Specifically, it improves the success rate from 72% to 93% on the benchmark dataset and from 20% to 76% on the industrial dataset.

The main contribution of this paper is to propose an automated framework for simultaneous multi-property clock-bound repair of NTA. MPTA-Repair combines iterative verification, a shared pool of property-trace constraints, relation-guided MaxSMT candidate generation, full-property regression verification, and an admissibility check based on functional equivalence. The evaluation against the TARTAR baseline shows improved repair success and identifies practical lessons on global validation and search-space limitations.

The remainder of the paper is organized as follows. Section II introduces the preliminaries and problem setting. Section III presents MPTA-Repair. Section IV reports the experimental evaluation. Section V concludes the paper.

II. Preliminaries

A. Networks of Timed Automata

The timed automaton model used in this paper follows the standard timed-automata semantics [5].

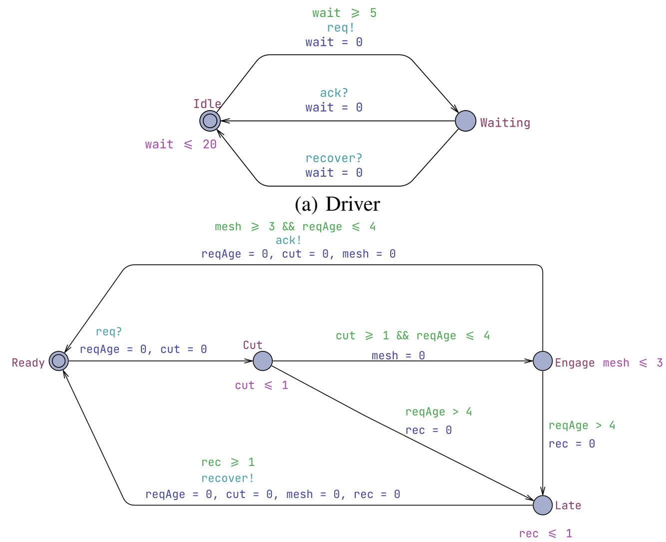
Definition 1. A timed automaton (TA) is a tuple

$$T = (L, \ell_0, C, \Sigma, \Theta, I) \tag{1}$$

where L is a finite set of locations, ℓ_0 is the initial location, C is a finite set of clocks, Σ is a set of action labels, Θ is a finite set of transitions, and I maps each location to a clock invariant. A clock constraint is a conjunction of atoms $x \sim b$, where $x \in C$, $\sim \in \{<, \leq, =, \geq, >\}$, and $b \in \mathbb{R}_{\geq 0}$. A transition $\theta = (\ell, g, a, R, \ell')$ moves from ℓ to ℓ' when guard g is satisfied, emits action a, and resets clocks in $R \subseteq C$. An NTA is a parallel composition $\mathcal{N} = T_1 \parallel \cdots \parallel T_k$ with the synchronization semantics of the target modeling language, such as UPPAAL [9]. Its behavior consists of action and delay transitions. Action transitions are instantaneous, require their guards to be satisfied, and reset the specified clocks; delay transitions model the passage of time in a location while respecting its invariant.

B. Timed Diagnostic Traces (TDTs) and Multi-Property Motivation

This paper focuses on timed safety properties, which are commonly used to specify deadlines, bounded waiting, and reachable error states in real-time systems. Such properties can be checked by tools such as UPPAAL: given an NTA and a property, the verifier either confirms the property or returns a counterexample trace [6].



(a) Driver

(b) GearController

Fig. 1. Simplified gear-shift example.

Definition 2. A symbolic timed trace (STT) of a model is a finite transition sequence $S = \theta_0, \ldots, \theta_{n-1}$. A realization of S is a sequence of non-negative delays $d_0, \ldots, d_n$ and states $s_0, \ldots, s_{n+1}$ such that, for every $i < n$, the model can delay from s_i by d_i and then take θ_i to s_{i+1} while respecting guards, resets, invariants, and synchronizations; the final delay d_n leads from s_n to s_{n+1}. The STT is feasible if it has at least one realization. Given a safety property φ, a feasible STT is a TDT for φ if at least one realization reaches a state that violates φ.

Example 1. Figure 1 shows the running gear-shift example. We check φ_1, `A[] (Controller.Engage imply cut >= 2)`, and φ_2, `A[] not Controller.Late`. In the faulty controller, the `Cut` invariant $cut \leq 1$ and the `Cut`→`Engage` guard $cut \geq 1$ allow engagement after one time unit, violating φ_1. A single-property repair raises both cut bounds to 2. If the `Engage` invariant $mesh \leq 3$ and the `Engage`→`Ready` guard $mesh \geq 3$ remain unchanged, the controller cannot return to `Ready` until $2 + 3 = 5$ time units after the request. It can therefore reach `Controller.Late` and violate φ_2. A multi-property repair must therefore adjust both cut and $mesh$ while preserving the original functional behavior.

TDTs provide concrete realizations of property violations, but they do not explain why the violations occur. In particular, a TDT does not identify which numerical bound is faulty or whether the relevant bound appears in a guard or an invariant. Following the idea of TARTAR [7], the repair procedure developed later in this paper encodes TDTs as arithmetic constraints over trace feasibility, property violation, and bound variations. Unlike single-property repair, MPTA-Repair organizes constraints from multiple violated properties and TDTs into a joint solving process to generate candidate clock-bound repairs.

Let M_{bad} be a faulty NTA model and let $\Phi = \{\varphi_1, \ldots, \varphi_m\}$ be the set of properties that must be satisfied simultaneously.

MPTA-Repair generates a repaired model M_{rep} by changing repairable numerical bound occurrences in guards and invariants. A candidate repair is accepted only if

$$M_{rep} \models \bigwedge_{\varphi \in \Phi} \varphi \quad \wedge \quad \mathrm{Accept}(M_{bad}, M_{rep}). \tag{2}$$

The first condition requires the repaired model to satisfy all required properties. The second condition denotes a TARTAR-style admissibility check based on functional equivalence [7]. This check prevents repairs that satisfy properties merely by disabling required actions.

III. Approach

A. Overview

MPTA-Repair takes an NTA model M and a property set $\Phi = \{\varphi_1, \ldots, \varphi_m\}$ as input. Instead of requiring manual variable specification, the framework extracts repairable bounds from numerical clock constraints in transition guards and location invariants. The final output is either a repaired model M' accompanied by concrete clock-bound edits, or a failure report if no valid full-property repair can be synthesized within the resource budget.

The framework operates on a local-generation and global-validation principle. While each TDT drives candidate generation, validation remains global to ensure that fixing one violation does not introduce regressions elsewhere. To leverage the interdependence where multiple violations share identical timing structures, MPTA-Repair maintains a joint pool of property-trace constraints. This joint structure enables the framework to exploit relations among properties, counterexamples, and repairable bounds to optimize the search space and guide the MaxSMT solver toward minimal parameter assignments.

Operationally, each refinement round begins by verifying M against Φ to collect TDTs for any violations. If violations are detected, the framework identifies active repairable bounds and constructs a joint MaxSMT problem from the active constraint pool. The synthesized parameter assignment is instantiated into a candidate model, which undergoes global validation via full-property regression verification and an admissibility check based on functional equivalence. A rejected candidate contributes newly exposed TDTs back into the pool to further constrain subsequent rounds, whereas a candidate passing both checks is returned as the valid repair M'. Figure 2 summarizes this counterexample-guided refinement loop from verification and relation-guided MaxSMT solving to global validation.

B. Joint Trace Encoding and Optimization Objective

MPTA-Repair follows the clock-bound variation strategy used in TARTAR [7]. Let S denote the set of repairable clock-bound occurrences. For each occurrence $s \in S$, the original numerical bound b_s in a transition guard or location invariant is associated with a variation variable δ_s. This ensures that the repaired constraint retains the original clock reference and comparison operator while incorporating a shifted value:

$$b'_s = b_s + \delta_s \tag{3}$$

The repair space is therefore strictly restricted to clock-bound constants, keeping transition structures and discrete updates unaltered.

MPTA-Repair builds on the trace-encoding formulation of TARTAR and extends it to the multi-property setting [7]. Given a TDT

$$\begin{aligned}\tau = \ell_0 \xrightarrow{d_0, e_0} \ell_1 \cdots \\ \xrightarrow{d_{n-1}, e_{n-1}} \ell_n\end{aligned} \tag{4}$$

a trace path is encoded as a conjunction of linear arithmetic constraints:

$$\begin{aligned}\mathrm{Path}(\tau, \rho, \mathbf{z}_\tau) = \mathit{Init}(\nu_0) \\ \wedge \bigwedge_{i=0}^{n-1} \Big(d_i \geq 0 \wedge \mathit{Inv}(\ell_i, \nu_i + d_i, \rho) \\ \wedge \mathit{Guard}(e_i, \nu_i + d_i, \rho) \\ \wedge \nu_{i+1} = \mathit{Reset}(\nu_i + d_i, R_i) \Big)\end{aligned} \tag{5}$$

Here, ρ denotes the modified bounds, and $\mathbf{z}_\tau$ collects the delays and clock valuations. The reset equation $\nu_{i+1} = \mathit{Reset}(\nu_i + d_i, R_i)$ updates the valuation after transition e_i by resetting clocks in R_i and preserving the others. The original violation encoding combines path executability with the final property violation:

$$\mathrm{Enc}(\tau, \varphi, \rho) = \mathrm{Path}(\tau, \rho, \mathbf{z}_\tau) \wedge \mathit{Bad}_\varphi(\ell_n, \nu_n) \tag{6}$$

The predicate $\mathit{Bad}_\varphi(\ell_n, \nu_n)$ characterizes the violation state of φ at the final location. To block this violating behavior while preserving executability of the discrete path skeleton, the single-trace repair constraint uses two copies of the trace variables:

$$\begin{aligned}\Psi^{\mathrm{repair}}_{\tau,\varphi}(\rho) = \exists \mathbf{z}^+_\tau . \, \mathrm{Path}(\tau, \rho, \mathbf{z}^+_\tau) \\ \wedge \neg \exists \mathbf{z}^-_\tau . \Big(\mathrm{Path}(\tau, \rho, \mathbf{z}^-_\tau) \\ \wedge \mathit{Bad}_\varphi(\ell^-_n, \nu^-_n) \Big)\end{aligned} \tag{7}$$

The first part requires that the original discrete path skeleton still has at least one timed realization after repair. The second part requires that no assignment of delays and clock valuations can make that same path violate φ.

We extend this single-trace formulation to a joint, multi-property incremental setting. At refinement round k, counterexamples from all currently violated properties are accumulated into a trace pool

$$\mathcal{T}_k = \{(\tau_j, \varphi_j)\}_{j=1}^{m_k} \tag{8}$$

MPTA-Repair assembles a joint MaxSMT problem with the constraint system defined as

$$\begin{aligned}\Phi_k(\rho) = \mathit{Dom}(\rho) \wedge \mathit{Block}_k(\rho) \\ \wedge \bigwedge_{(\tau,\varphi) \in \mathit{Select}(\mathcal{T}_k)} \Psi^{\mathrm{repair}}_{\tau,\varphi}(\rho)\end{aligned} \tag{9}$$

Here, $\mathit{Dom}(\rho)$ enforces static interval consistency and domain limits, such as $b'_s \geq 0$, and $\mathit{Select}(\mathcal{T}_k)$ represents a reduced

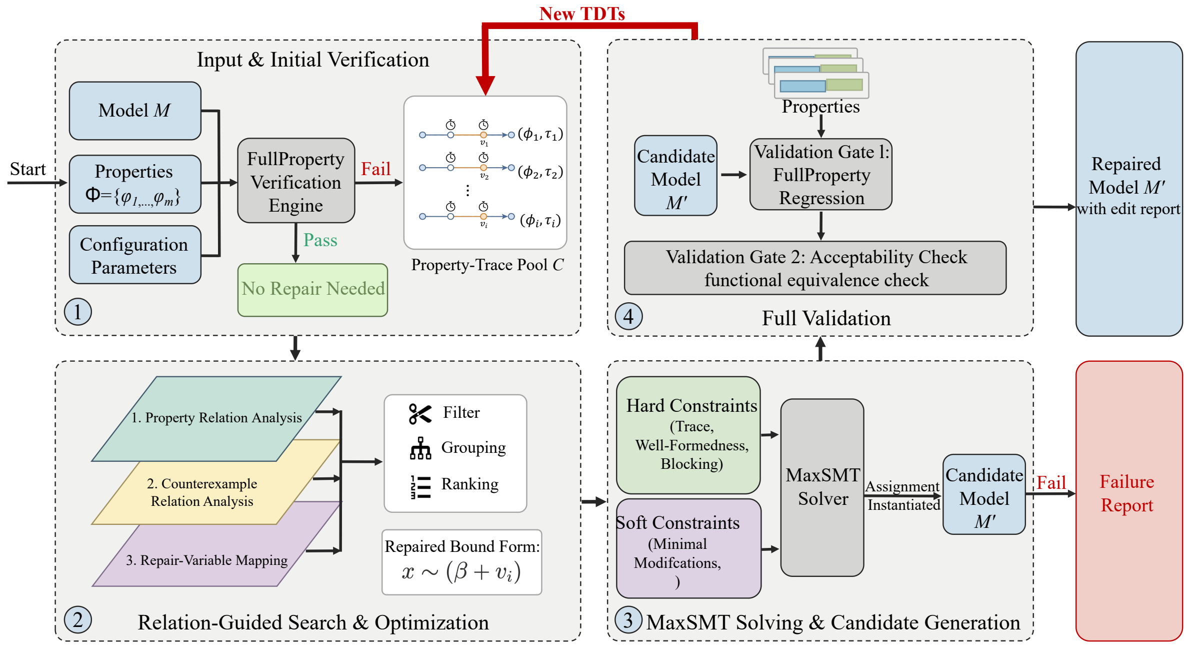


Fig. 2. Overview of the MPTA-Repair workflow.

subset of traces obtained after relation-aware filtering. When an instantiated candidate model fails verification, the constraint system is updated incrementally. The framework expands the pool via

$$\mathcal{T}_{k+1} = \mathcal{T}_k \cup NewTDTs(M(\rho_k), \Phi) \tag{10}$$

and updates the search-space block to exclude the failed assignment ρ_k over the set of repairable bounds S:

$$Block_{k+1}(\rho) = Block_k(\rho) \wedge \left(\bigvee_{s \in S} \delta_s \neq \delta_s^k \right) \tag{11}$$

where δ_s^k is the valuation assigned to variation variable δ_s in round k.

The optimization objective is governed by a hierarchical set of soft constraints. MPTA-Repair uses a lexicographic objective that first favors small repairs and then incorporates risk- and coverage-based priorities:

$$\operatorname*{lex\,min}_{\rho} \begin{pmatrix} \sum_{s \in S} \mathbf{1}\{\delta_s \neq 0\}, \\ \sum_{s \in S} |\delta_s|, \\ \sum_{s \in S} risk_s \cdot \mathbf{1}\{\delta_s \neq 0\}, \\ -\sum_{s \in S} benefit_s \cdot \mathbf{1}\{\delta_s \neq 0\} \end{pmatrix} \tag{12}$$

Here, $\mathbf{1}\{\cdot\}$ denotes the indicator function, yielding 1 when the inner condition holds and 0 otherwise. The primary and secondary objectives minimize the number and total magnitude of altered clock bounds to produce concise repairs. The third term incorporates a penalty coefficient $risk_s$ to discourage modifications prone to regression, while the final term uses a reward coefficient $benefit_s$ to prioritize repair sites that cover a broader set of violated traces.

C. Relation-Guided Search Optimization

During refinement, accumulating violated properties, TDTs, and editable bounds expands the MaxSMT search space. To reduce redundant constraints, MPTA-Repair uses relation-guided optimization to filter, group, and rank constraints and repair variables. These heuristics only guide the search; correctness is still guaranteed through full-property regression verification and an admissibility check.

At the property layer, timed safety properties are normalized into an `A[]` fragment over location predicates and clock or integer constraints. Equivalence is detected by the syntactic identity of normalized forms, while dominance is evaluated under logical subsumption, e.g., `A[]` $x \leq 5$ subsumes `A[]` $x \leq 10$ within the same structural context. MPTA-Repair then retains only the stronger properties for solving to optimize efficiency.

At the counterexample layer, MPTA-Repair maintains a TDT pool indexed by violated properties. Exact duplicates are removed, and the remaining traces are annotated with traversed transitions, visited locations, active clocks, and guard or invariant bounds. Traces with identical discrete transition sequences or overlapping clock zones are grouped to reveal common bottlenecks. A trace constraint is omitted from the active encoding only when it is an exact duplicate or soundly subsumed by an encoded trace constraint.

At the repair-bound layer, MPTA-Repair maps each active property-trace constraint to the clock bounds occurring in or affecting its encoding. This mapping restricts active MaxSMT decision variables to bounds relevant to the current counterexample pool and ranks them for candidate generation. Bounds with higher relational scores are prioritized because their modifications are more likely to satisfy the joint constraint system.

D. Validation, Admissibility, and Refinement

In MPTA-Repair, candidate validation determines whether a generated edit set can be accepted as a repair. After a candidate edit set is generated, the repaired model is verified against the entire property set Φ. If a candidate resolves the observed violation but induces new violations against the specification, it is rejected, indicating that the current property-trace pool is insufficient. MPTA-Repair then extracts the newly observed TDTs and appends them to the pool, guiding the subsequent refinement round.

Candidates that pass full-property regression verification are further evaluated against an admissibility check based on functional equivalence [7]. This step ensures that clock-bound modifications do not satisfy properties in ways that fail to preserve the original functional behavior. Rather than enforcing timed-language equivalence, which would conflict with the goal of eliminating violating timed paths, the check is performed by constructing untimed automata from both the original and repaired models and comparing their accepted untimed languages [18]. A candidate is rejected if it adds or removes discrete functional paths, ensuring that only timing characteristics are modified.

The refinement loop therefore combines local candidate generation with global validation. Local MaxSMT formulations propose candidate assignments, full-property regression verification ensures complete specification compliance, and the admissibility check preserves the original functional behavior.

IV. Experimental Evaluation

This section evaluates MPTA-Repair on both benchmark and industrial case studies. The evaluation follows the clock-bound repair setting defined in Section III, where only numerical bounds in guards and invariants are repairable. For comparison, we evaluate MPTA-Repair against an adapted iterative TARTAR-style baseline [7].

A. Evaluation Strategy and Datasets

We evaluate MPTA-Repair on two datasets, summarized in Table I. The benchmark dataset includes nine benchmark models derived from UPPAAL benchmark [16] and the XtaBenchmarkSuite [17], such as Elevator and FDDI. The industrial dataset constructed in this work contains five closely coupled NTA case-study models: Gear Control System for automotive transmission [19], SAE RoR for autonomous-driving decisions [20], Train Controller Alarm for railway emergency timing [21], Train Gate Controller for shared-resource concurrency [4], and Pacemaker for medical pacing [8]. Compared with the benchmark models, these case studies involve more tightly coupled timing constraints across multiple properties.

Since real-world NTA models with labeled clock-bound faults are difficult to obtain, we synthesize faulty instances using mutation [22], retaining only mutants that are syntactically valid, violate at least one timing safety property, and preserve the original functional behavior. Each repair instance is executed with a 10-minute timeout for both MPTA-Repair and the baseline.

TABLE I
Statistics of the benchmark and industrial datasets.

Dataset	Original models	Mutated models	Avg. properties/model
Benchmark	9	54	4.1
Industrial	5	105	7.3

TABLE II
Repair results across benchmark and industrial datasets.

Dataset	Method	Mutated models	Repaired	Success rate	Avg. time
Benchmark	Iterative baseline	54	39	72%	60.61s
	MPTA-Repair	54	50	93%	15.46s
Industrial	Iterative baseline	105	21	20%	35.59s
	MPTA-Repair	105	80	76%	85.28s

We compare MPTA-Repair against an adapted iterative TARTAR-style baseline [7], which was originally designed for single-property repair and operates via trace-local repair generation. This comparison allows us to examine whether using a joint counterexample pool improves repair robustness over localized trace analysis.

B. Experimental Results

The experimental evaluation demonstrates that MPTA-Repair outperforms the iterative baseline, particularly on complex industrial models. As shown in Table II, the baseline achieves a 72% success rate on the benchmark dataset, whereas MPTA-Repair reaches 93%. This performance gap widens on the industrial dataset, where the baseline repairs only 20% of the cases compared to 76% for MPTA-Repair. The baseline reports a lower average duration on the industrial dataset because unsuccessful runs typically terminate early when no valid candidate can be found.

The baseline remains competitive on simpler benchmarks, where injected timing faults are often localized to a small number of guards or invariants. However, its performance drops significantly on the industrial dataset, where, under concurrent component interactions and tightly coupled timing constraints, trace-local repairs often produce candidates that fail full-property regression verification or the admissibility check.

MPTA-Repair achieves higher success rates by integrating constraints from TDTs and properties into a joint generation process. Consequently, the framework produces coordinated modifications that pass both full-property regression verification and the admissibility check.

C. Discussion and Practical Lessons

The evaluation highlights two primary lessons for applying automated multi-property repair to NTA. First, full-property regression verification is necessary because industrial properties are often interdependent; a localized repair for one response-time property can inadvertently violate another safety property, such as maximum waiting bounds. MPTA-Repair avoids these regressions by requiring each candidate to satisfy the full property set. Second, the admissibility check is critical to distinguish valid repairs from behavior-breaking edits that satisfy timing properties only by removing intended functional behavior, such as preventing critical system actions. The admissibility check ensures that property satisfaction is not achieved at the expense of the original functional behavior.

Nevertheless, the framework can still fail to generate repairs in certain cases when it fails to find an admissible candidate within the time budget. This limitation is prominent in complex models like the Pacemaker case study, where tightly coupled timing constraints expand the search space, causing the solver to time out before finding a valid candidate.

V. Conclusion

This paper presented MPTA-Repair, an automated workflow for multi-property and multi-counterexample clock-bound repair of industrial NTA models. To overcome the limitations of isolated single-trace fixes, the method accumulates TDTs and exploits relations among properties, counterexamples, and repairable bounds to guide MaxSMT-based search. Each candidate is validated through full-property regression verification and an admissibility check based on functional equivalence [7], ensuring that accepted repairs preserve the original functional behavior.

Evaluations across benchmark and industrial datasets show that MPTA-Repair outperforms the iterative baseline, particularly in industrial systems with dense property interactions. The failure analysis further indicates that unsuccessful repairs mainly arise when the framework cannot find an admissible candidate within the time budget, especially as dense timing interactions enlarge the clock-bound repair search space.

Future work will primarily focus on extending the automated repair space beyond numerical clock bounds to include broader structural modifications, such as altering synchronization labels and redesigning discrete control logic. We also plan to integrate the admissibility check earlier in repair generation, so that functionally invalid candidates can be pruned before costly validation and large-scale industrial repair can be made more efficient.

References

[1] D. Basile, A. Fantechi, and I. Rosadi, "Formal analysis of the UNISIG safety application intermediate sub-layer: Applying formal methods to railway standard interfaces," in *Proceedings of Formal Methods for Industrial Critical Systems*, ser. Lecture Notes in Computer Science, vol. 12863. Springer, 2021, pp. 174–190.

[2] H. Lönn and P. Pettersson, "Formal verification of a TDMA protocol start-up mechanism," in *Proceedings of the 1997 IEEE Pacific Rim International Symposium on Fault-Tolerant Systems*, 1997, pp. 235–242.

[3] M. Mikučionis, K. G. Larsen, J. I. Rasmussen, B. Nielsen, A. Skou, S. U. Palm, J. S. Pedersen, and P. Hougaard, "Schedulability analysis using Uppaal: Herschel-planck case study," in *Proceedings of Leveraging Applications of Formal Methods, Verification and Validation*, ser. Lecture Notes in Computer Science, vol. 6416. Springer, 2010, pp. 175–190.

[4] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.

[5] J. Bengtsson and W. Yi, "Timed automata: Semantics, algorithms and tools," in *Lectures on Concurrency and Petri Nets*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3098, pp. 87–124.

[6] T. Vogel, M. Carwehl, G. N. Rodrigues, and L. Grunske, "A property specification pattern catalog for real-time system verification with UPPAAL," 2022, arXiv:2211.03817.

[7] M. Kölbl, S. Leue, and T. Wies, "Clock bound repair for timed systems," in *International Conference on Computer Aided Verification*. Springer, 2019, pp. 79–96.

[8] Z. Jiang, M. Pajic, A. Connolly, S. Dixit, and R. Mangharam, "Real-time heart model for implantable cardiac device validation and verification," in *Proceedings of the Euromicro Conference on Real-Time Systems*. IEEE Computer Society, 2010, pp. 239–248.

[9] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997.

[10] C. Daws, A. Olivero, S. Tripakis, and S. Yovine, "The tool KRONOS," in *Hybrid Systems III*, ser. Lecture Notes in Computer Science. Springer, 1996, vol. 1066, pp. 208–219.

[11] A. E. Dalsgaard, R. R. Hansen, K. Y. Jørgensen, K. G. Larsen, M. C. Olesen, P. Olsen, and J. Srba, "opaal: A lattice model checker," in *NASA Formal Methods*, ser. Lecture Notes in Computer Science, vol. 6617. Springer, 2011, pp. 487–493.

[12] R. Alur, T. A. Henzinger, and M. Y. Vardi, "Parametric real-time reasoning," in *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*. ACM, 1993, pp. 592–601.

[13] Étienne André, P. Arcaini, A. Gargantini, and M. Radavelli, "Repairing timed automata clock guards through abstraction and testing," in *Proceedings of Tests and Proofs*, ser. Lecture Notes in Computer Science, vol. 11823. Springer, 2019, pp. 129–146.

[14] J. Bendík, A. Sencan, E. A. Gol, and I. Černá, "Timed automata robustness analysis via model checking," *Logical Methods in Computer Science*, vol. 18, no. 3, pp. 12:1–12:32, 2022.

[15] M. Jose and R. Majumdar, "Bug-assist: Assisting fault localization in ANSI-C programs," in *Proceedings of Computer Aided Verification*, ser. Lecture Notes in Computer Science, vol. 6806. Springer, 2011, pp. 504–509.

[16] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on Uppaal," in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science, vol. 3185. Springer, 2004, pp. 200–236.

[17] R. Farkas and G. Bergmann, "Towards reliable benchmarks of timed automata," in *Proceedings of the 25th PhD Mini-Symposium*, 2018, pp. 20–23.

[18] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 2nd ed. Addison-Wesley, 2001.

[19] M. Lindahl, P. Pettersson, and W. Yi, "Formal design and analysis of a gear controller," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, vol. 1384. Springer, 1998, pp. 281–297.

[20] G. V. Alves, L. A. Dennis, and M. Fisher, "A double-level model checking approach for an agent-based autonomous vehicle and road junction regulations," *Journal of Sensor and Actuator Networks*, vol. 10, no. 3, p. 41, 2021.

[21] A. González, M. Cristiá, and C. Luna, "Verification of quantitative temporal properties in RealTime-DEVS," *SIMULATION: Transactions of The Society for Modeling and Simulation International*, vol. 101, no. 10, pp. 1057–1076, 2025.

[22] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.